Experiment 3

Day-Time Client–Server Application

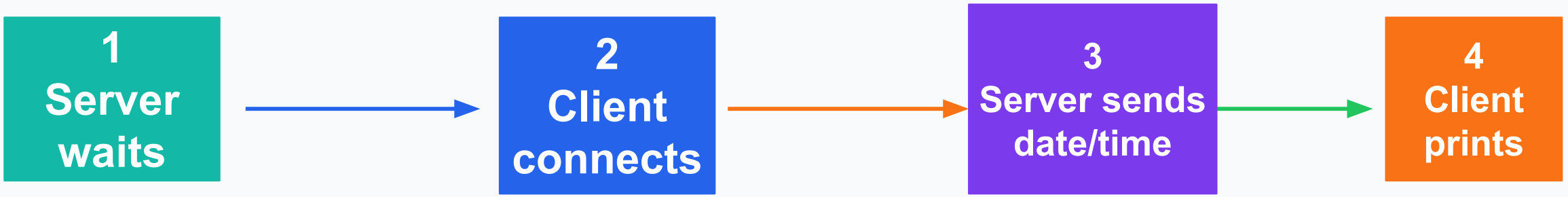


Connect → Receive date/time → Display output

Aim of the Program

Aim

Implement a Java TCP day-time client–server application.



In your practical code, both programs use port 5217. The port must match on both sides.

What Is Happening?



Why? The experiment shows how two separate Java programs communicate over a network.

The date/time is just example data. The real topic is client–server communication.

Server vs ServerSocket

SERVER

Complete program/system that provides a service

contains

ServerSocket

predefined Java class

accept()

Socket soc: actual connection object actual communication

Restaurant example

Restaurant = SERVER
Reception desk = ServerSocket
Table/conversation with customer = Socket

```
class DateServer {
  ServerSocket s = new
ServerSocket(5217);
  Socket soc = s.accept();
}
```

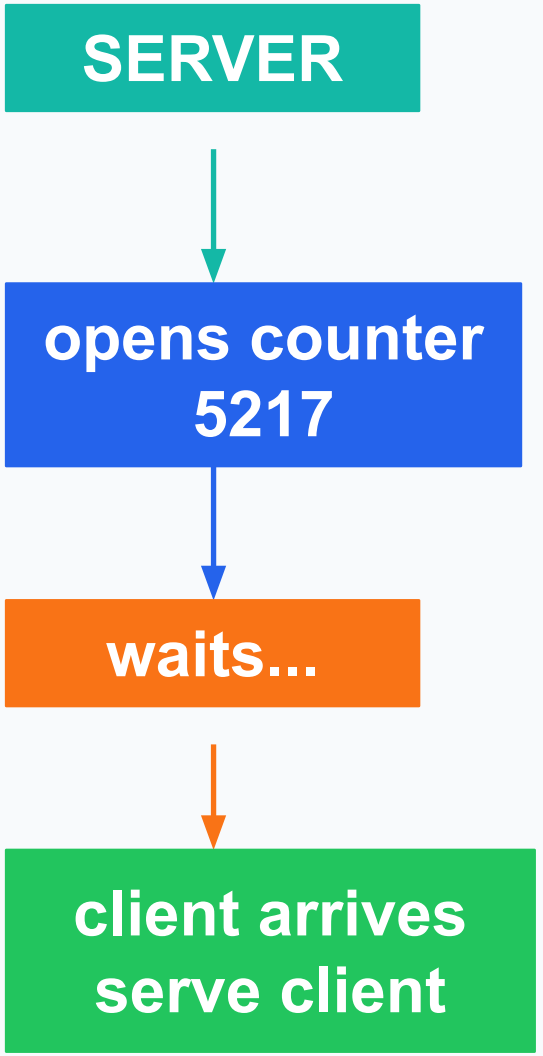
Server = complete service provider; ServerSocket = listening part inside the server; Socket = connection with one client.

Why Does the Server Wait?

```
ServerSocket s = new
ServerSocket(5217);
Socket soc = s.accept();
```

The server does not know when a client will come, so it starts first and keeps listening on port 5217.

accept() means: “wait here until someone connects.”



Why Is TCP Used?

TCP stands for Transmission Control Protocol. It creates a connection before data is exchanged.



This matches the program: client connects → server accepts → server sends date/time → client receives → connection closes.

```
ServerSocket + Socket  →  TCP connection
```

Why TCP Instead of UDP?

For this practical, we want the complete date/time line to reach the client correctly.

Reliable delivery

TCP tries to ensure data reaches the other side, or reports failure.

Correct order

Sent: A B C D
Received: A B C D

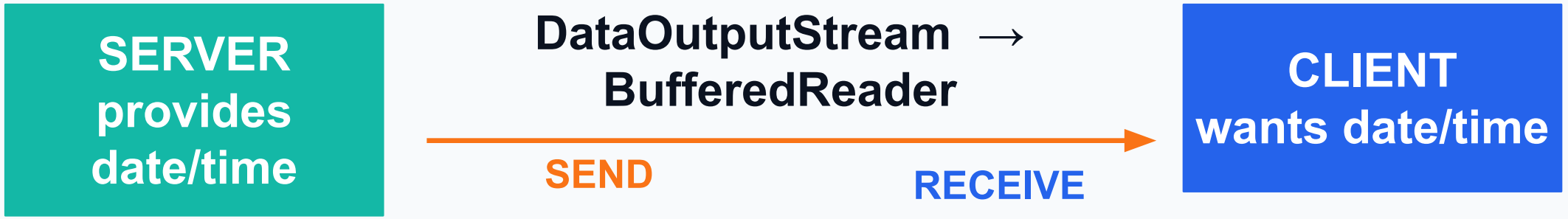
Connection-oriented

Client and server connect first, then exchange data.

```
Server Date: Thu Aug 13 18:50:30 IST 2026
```

Why Server Sends and Client Receives

Their roles are different: the server provides the service, and the client uses the service.




```
Server: soc.getOutputStream()  // sending (outgoing path)
Client: soc.getInputStream()   // receiving (incoming path)
```

Full Server Code: DateServer.java

SERVER CODE

```
 01  import java.net.*;
  02  import java.io.*;
  03  import java.util.*;
  04  class DateServer
  05  {
  06    public static void main(String args[]) throws Exception
  07    {
>> 08      ServerSocket s = new ServerSocket(5217);
>> 09      while(true)
  10      {
  11        System.out.println("Waiting For Connection ...");
>> 12        Socket soc = s.accept();
>> 13        DataOutputStream out = new DataOutputStream(soc.getOutputStream());
>> 14        out.writeBytes("Server Date: "+(new Date()).toString()+"\n");
  15        out.close();
  16        soc.close();
  17      }
  18    }
  19  }
```

>> lines are the main working part of the server.

Server Code: Imports + Class

`import java.net.*;`	Loads networking classes such as ServerSocket and Socket.
`import java.io.*;`	Loads input/output stream classes used to send data.
`import java.util.*;`	Loads utility classes. Here it is needed for Date.
`class DateServer`	Defines the server-side program class.
`{`	Opening brace: body of DateServer starts here.

Server Code: main() + Port Binding

```
public static void
main(String args[]) throws
Exception
```

Program execution starts here. throws Exception handles possible network errors.

```
{
```

Opening brace: main method body starts.

```
ServerSocket s = new
ServerSocket(5217);
```

Creates the server and binds it to port 5217.

```
ServerSocket = waiting desk
Port 5217 = door number
Client must use same port
```
Binds: Reserve that counter for this server



Server Code: Loop + accept()

```
while(true)
```

Keeps the server running continuously.

```
{
```

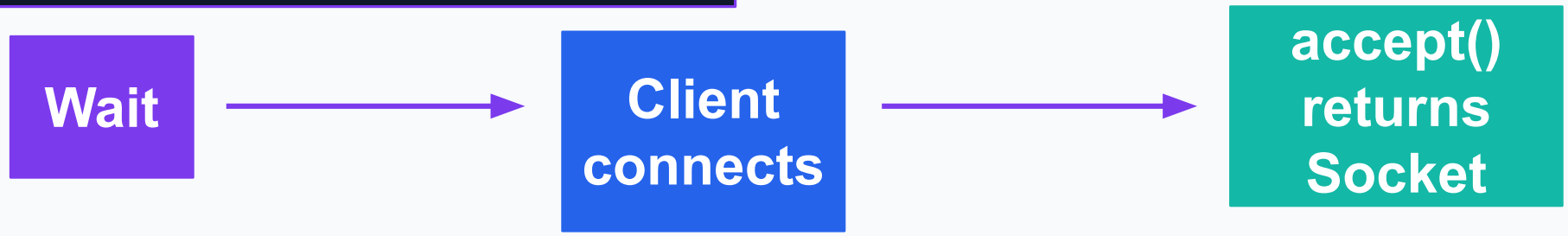
Opening brace of the loop.

```
System.out.println("Waiting For Connection ...");
```

Shows message on server console while waiting.

```
Socket soc = s.accept();
```

Waits for a client. When a client connects, it creates a Socket.



Experiment 3

Server Code: Send Date/Time main sending part of the server.

```
DataOutputStream out =
new
DataOutputStream(soc.get
OutputStream());
```

Creates an output stream to send data from server to client. Give me the outgoing path of this socket.

```
out.writeBytes("Server
Date: "+(new
Date()).toString()+"\n")
;
```

Take the outgoing channel of the connected socket, wrap it in a DataOutputStream, and store that sending object in a variable called out.

Gets current date/time, converts it to text, and sends it to the client.

```
new Date()
    ↓
convert to String
    ↓
writeBytes()
```



Experiment 3

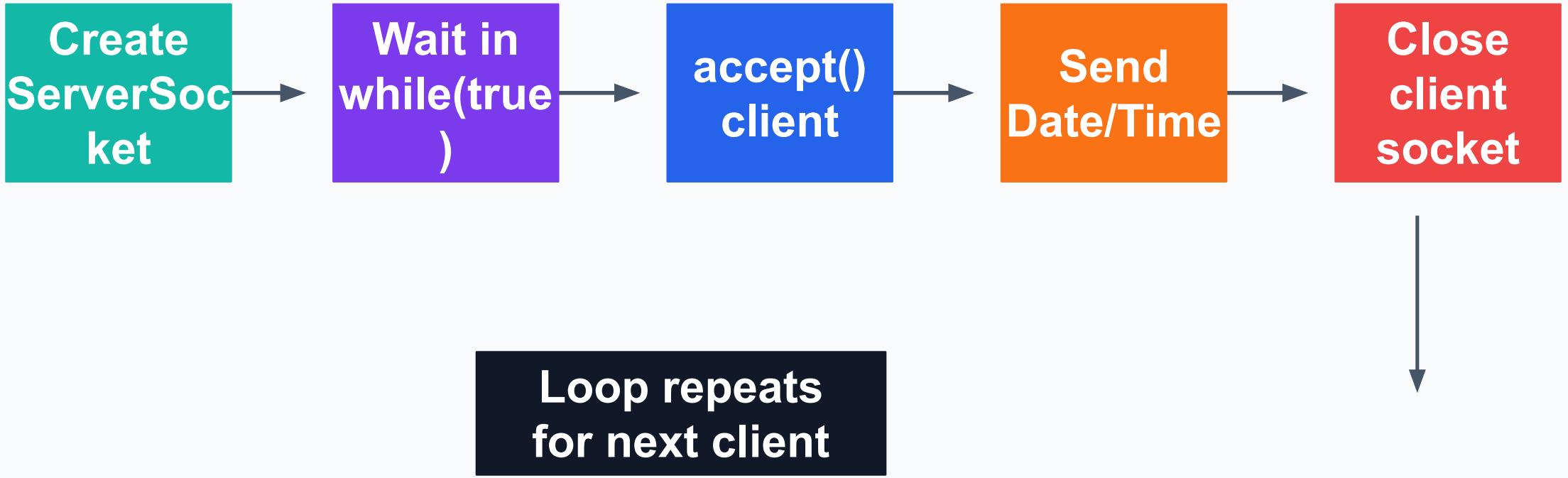
Server Code: Close + Repeat

`out.close();`	Closes the output stream after sending date/time.
`soc.close();`	Closes the current client connection.
`}`	Ends the while loop block. Because while(true) is used, it goes back and waits again.
`}`	Ends the main method.
`}`	Ends the DateServer class.

Important: serverSocket is not closed inside the loop, because the server must keep waiting for new clients.

Experiment 3

Server Flow Diagram



This is why the server can serve many clients one after another.

Full Client Code: DateClient.java

CLIENT CODE

```
 01  import java.io.*;
  02  import java.net.*;
  03  class DateClient
  04  {
  05    public static void main(String args[]) throws Exception
  06    {
>> 07      Socket soc = new Socket(InetAddress.getLocalHost(), 5217);
>> 08      BufferedReader in = new BufferedReader(new
InputStreamReader(soc.getInputStream()));
>> 09      System.out.println(in.readLine());
  10    }
  11  }
```

>> lines are the main working part of the client.

Client Code: Imports + Connection

```
import java.io.*;
```

Loads input/output classes for reading data from server.

```
import java.net.*;
```

Loads networking classes: Socket and InetAddress.

```
class DateClient
```

Defines the client-side program class.

```
public static void
main(String args[]) throws
Exception
```

Client program starts here.

```
Socket soc = new
Socket(InetAddress.getLocalHo
st(), 5217);
```

Connects to the server running on the same machine at port 5217. **Socket = communication connection**

InetAddress.getLocalHost() means: connect to the server running on this computer.

Experiment 3

Client Code: Receive + Print

```
BufferedReader in = new
BufferedReader(new
InputStreamReader(soc.getIn
putStream()));
```

Creates a reader to receive text from the server through the socket.
getInputStream() = receive bytes →
InputStreamReader = bytes to text →
BufferedReader = read text easily → in = name of the reader.

```
soc.getInputStream()
```

Gets the incoming data stream from the server.

```
InputStreamReader(...)
```

Converts bytes from the network into characters.

```
BufferedReader(...)
```

Makes it easy to read a full line using readLine().

```
System.out.println(in.re
adLine());
```

Reads one line sent by the server and prints it on client console.

Experiment 3

Same Port Rule

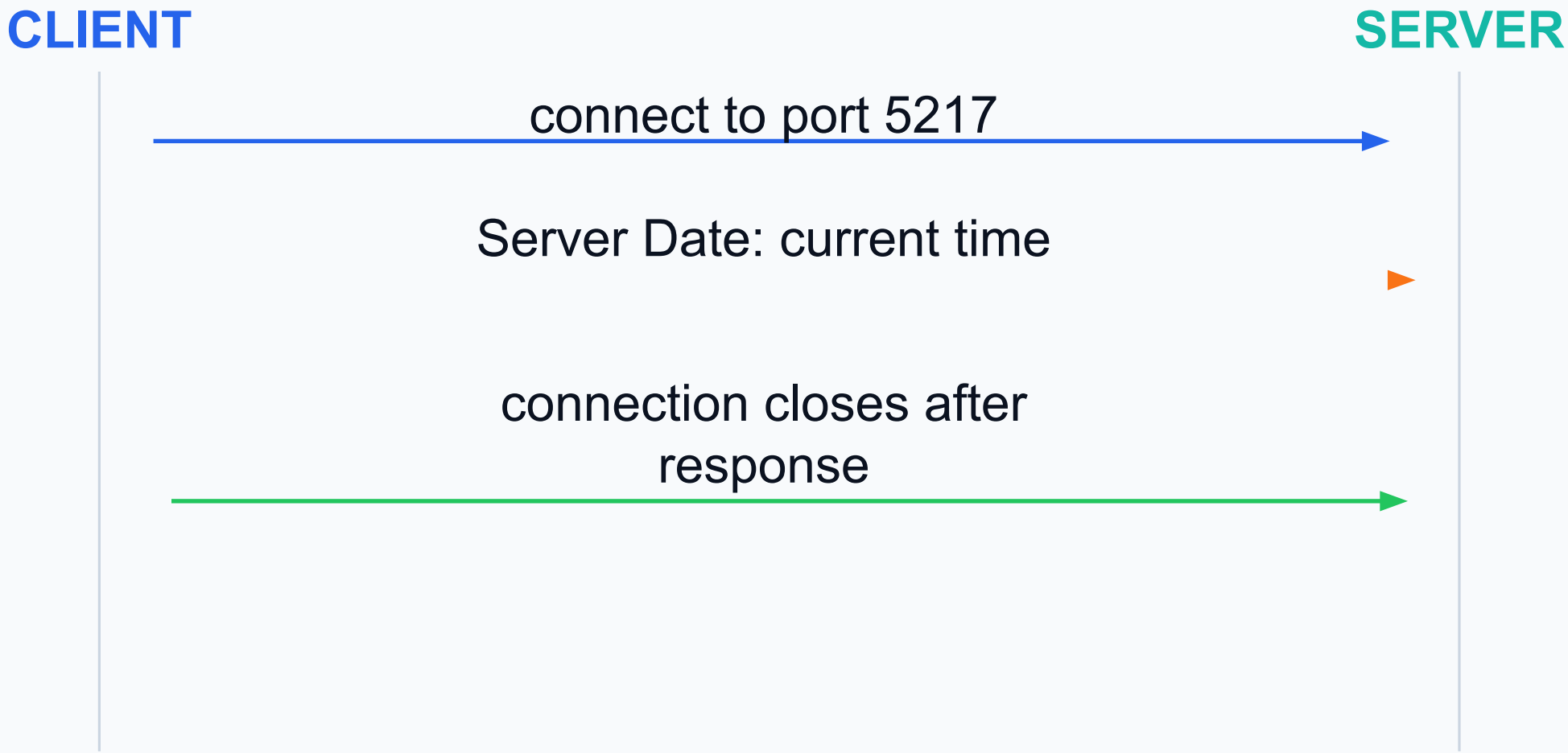
Both sides must use the same port number.



If the server uses 5217 but the client tries 5000, connection will fail.

Experiment 3

Complete Communication Sequence



1. Server starts first. 2. Client connects. 3. Server sends date/time. 4. Client prints output.

How to Run the Program

1. Compile both files:
   javac DateServer.java
   javac DateClient.java

2. Run the server first:
   java DateServer

3. Run the client in another terminal:
   java DateClient

```
Server console:
Waiting For Connection
...

Client console:
Server Date: Thu Aug 13
18:50:30 IST 2026
```

Always start the server before the client, otherwise the client may show connection refused.

Java Parts: Listen and Connect

Very simple meaning of the first four networking parts

`ServerSocket`

WHAT: Server waits on a port.
WHY: Without it, no client can find the server.

`Socket`

WHAT: Actual connection between client and server.
WHY: Both sides need a path to talk.

`accept()`

WHAT: Server pauses and waits for a client.
WHY: Client may come at any time.



Memory line: ServerSocket = waits, Socket = talks, Port = correct door.

Java Parts: Sending Data

These parts are used on the SERVER side to prepare and send the date/time.

`DataOutputStream`

WHAT: A sending tool.
WHY: Server uses it to send response to client.

`getOutputStream()`

WHAT: Outgoing channel.
WHY: Data must leave server through this path.

`writeBytes()`

WHAT: Writes text to the network.
WHY: Actually sends the server response.

new Date() → toString() → writeBytes() → Client receives

```
out.writeBytes("Server Date: "+(new Date()).toString()+"\n");
```

Java Parts: Receiving Data

These parts are used on the CLIENT side to receive and read server data.

`getInputStream()`

WHAT: Incoming channel.
WHY: Client must receive server data through it.
Example: Inbox receiving LinkedIn data.

`InputStreamReader`

WHAT: Converts bytes into characters.
WHY: Network gives bytes, but humans read text.
Example: Translator: bytes → text.

`BufferedReader`

WHAT: Reads text easily.
WHY: It provides readLine().
Example: A neat reader for one full line.




```
System.out.println(in.readLine());
```

Java Parts: Cleanup + Full Memory Table

close()

WHAT: closes stream/socket. WHY: communication for that client is finished, so resources should be released.

Java Part	WHAT	WHY
`ServerSocket`	Listens	server waits for clients
`Socket`	Connection	client/server talk through it
`accept()`	Waits	client may come any time
`Port 5217`	Door	same destination on both sides
`OutputStream`	Send	data leaves server
`InputStream`	Receive	data enters client
`writeBytes()`	Writes	sends response
`readLine()`	Reads	receives one line

Final memory line: Server waits → Client connects → Server sends → Client receives → Both close.